

1. Цель и задачи

Цель: изучить принципы построения цветовых моделей RGB, CMYK, HLS и разработать веб-приложение для интерактивного выбора и взаимного пересчёта цвета в режиме реального времени.

Задачи:

1. Реализовать точные формулы взаимного перевода между RGB, CMYK и HLS.
2. Создать графический интерфейс с тремя способами управления цветом: числовые поля, ползунки (слайдеры) и палитра.
3. Обеспечить автоматический пересчёт всех компонентов при изменении любого параметра.

2. Теория

RGB в CMYK, при нормализованных $r, g, b \in [0, 1]$:

$$k = 1 - \max(r, g, b)$$

$$c = \frac{1 - r - k}{1 - k}, \quad m = \frac{1 - g - k}{1 - k}, \quad y = \frac{1 - b - k}{1 - k}$$

$$c = m = y = 0, \text{ если } k = 1$$

CMYK в RGB:

$$r = (1 - c)(1 - k), \quad g = (1 - m)(1 - k), \quad b = (1 - y)(1 - k)$$

RGB в HLS:

$$\Delta = \max(r, g, b) - \min(r, g, b)$$

$$S = \begin{cases} 0 & \text{если } \Delta = 0 \\ \frac{\Delta}{1 - |2L - 1|} & \text{если } \Delta \neq 0 \end{cases}$$

$$L = \frac{\max(r, g, b) + \min(r, g, b)}{2}$$

Значение цветового тона $H \in [0^\circ, 360^\circ)$ определяется угловым сектором преобладающего канала.

3. Ход работы

Архитектура: В качестве универсального промежуточного базиса выбрана модель RGB. Любое изменение в UI нормализуется в RGB, после чего синхронно пересчитываются координаты для CMYK и HLS.

Интерфейс и элементы управления:

- Созданы поля числового ввода и ползунки с валидацией допустимых диапазонов (0..255, 0..100, 0..360).
- Добавлена контрольная плашка для наглядного отображения результирующего цвета.

Особенности реализации палитры:

- Графический селектор на HTML5 Canvas выполнен по аналогии с интерфейсом Krita (внешнее кольцо тона H и внутренний треугольник оттенков).
- В отличие от прямоугольного селектора, представляющего собой плоский срез цилиндрической модели (где чёрный или белый цвета вырождаются в целые рёбра и создают избыточные области), треугольная модель сводит чистый тон, белый и чёрный полюса к трём вершинам.
- Расчёт положения точки внутри треугольника с помощью барицентрических координат обеспечивает естественное художественное смешивание тонов без геометрических искажений.

Синхронизация: Функция `update()` диспетчеризует обновления, синхронизирует состояния элементов ввода и перерисовывает маркеры палитры без взаимной блокировки и зацикливания событий.

КГ. Лаба 1. RGB - CMYK - HLS

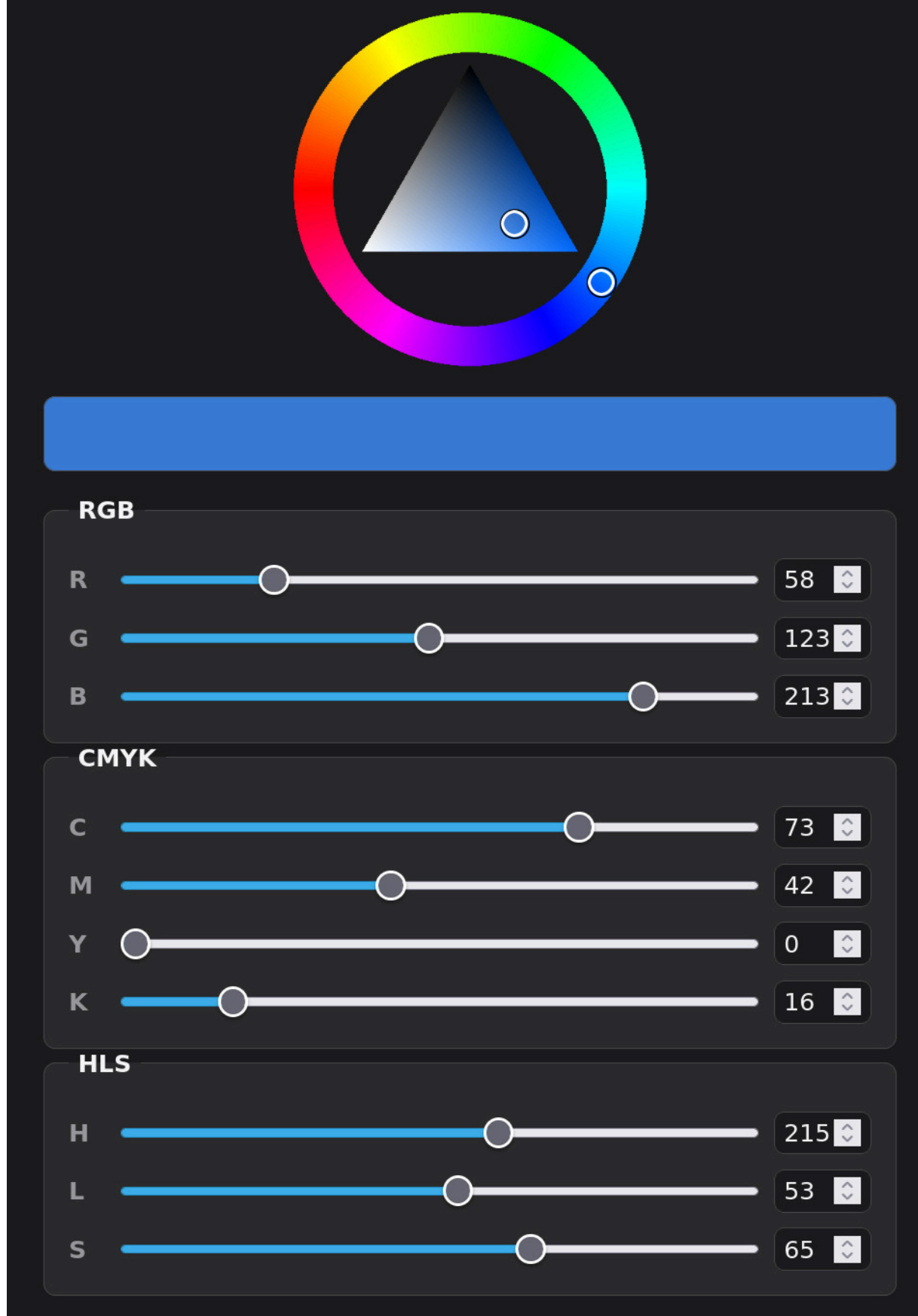


Рисунок 1. Интерфейс приложения

4. Выводы

- Изучены ключевые особенности цветовых пространств RGB, CMYK и HLS, а также их специфика в компьютерной графике.
- Разработано веб-приложение, полностью удовлетворяющее требованиям ТЗ: корректный математический пересчёт между моделями, три способа ввода и автоматическая синхронизация компонентов.
- Интеграция треугольного цветового селектора в стиле Krita позволила объединить строгую математическую модель преобразований с эргономичным интерфейсом выбора цвета.

5. Листинг кода программы

```
<!DOCTYPE html>
<html lang="ru">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1, viewport-fit=cover">
  <title>КГ. Лаба 1. RGB - CMYK - HLS</title>
  <style>
    :root {
      --bg: #1c1c1e;
      --panel: #2c2c2e;
      --text: #f5f5f7;
      --muted: #98989d;
      --line: #48484a;
      box-sizing: border-box;
      padding-top: env(safe-area-inset-top, 0px);
      padding-bottom: env(safe-area-inset-bottom, 0px);
    }

    html {
      scroll-padding-top: env(safe-area-inset-top, 0px);
    }

    body {
      margin: 0;
      background: var(--bg);
      color: var(--text);
      font-family: system-ui, -apple-system, "Segoe UI", Roboto,
sans-serif;
    }

    main {
      max-width: 560px;
      margin: 0 auto;
      padding: 24px 16px;
```

```

        display: grid;
        gap: 16px;
    }

    h1 {
        font-size: 22px;
        margin: 0;
    }

    #colorBar {
        height: 48px;
        border: 1px solid var(--line);
        border-radius: 8px;
    }

    #wheel {
        position: relative;
        width: 240px;
        height: 240px;
        margin: 0 auto;
        cursor: crosshair;
        touch-action: none;
    }

    #wheel canvas {
        display: block;
        width: 240px;
        height: 240px;
    }

    .marker {
        position: absolute;
        width: 14px;
        height: 14px;
        margin: -7px 0 0 -7px;
        border: 2px solid #fff;
        border-radius: 50%;
        box-shadow: 0 0 0 1px #000;
        pointer-events: none;
    }

    fieldset {
        margin: 0;
        padding: 12px 16px 16px;
        background: var(--panel);
        border: 1px solid var(--line);
        border-radius: 8px;
    }

    legend {
        font-weight: 600;
        padding: 0 6px;
    }

```

```

    .row {
      display: grid;
      grid-template-columns: 20px 1fr 64px;
      align-items: center;
      gap: 12px;
      margin-top: 10px;
    }

    .row span {
      color: var(--muted);
      font-weight: 600;
    }

    .row input[type="range"] {
      width: 100%;
      min-width: 0;
    }

    .row input[type="number"] {
      width: 100%;
      box-sizing: border-box;
      padding: 4px 6px;
      border: 1px solid var(--line);
      border-radius: 6px;
      background: var(--bg);
      color: var(--text);
      font: inherit;
    }
  </style>
</head>

<body>
  <main>
    <h1>КГ. Лаба 1. RGB - CMYK - HLS</h1>
    <div id="wheel">
      <canvas id="canvas" width="240" height="240"></canvas>
      <div id="hueMarker" class="marker"></div>
      <div id="colorMarker" class="marker"></div>
    </div>
    <div id="colorBar"></div>
    <div id="models"></div>
  </main>
  <script>
    const MODELS = {
      RGB: {
        labels: ["R", "G", "B"],
        max: [255, 255, 255],
        toRgb: v => v,
        fromRgb: v => v
      },
      CMYK: {
        labels: ["C", "M", "Y", "K"],
        max: [100, 100, 100, 100],
        toRgb([c, m, y, k]) {

```

```

        return [c, m, y].map(x => 255 * (1 - x / 100) * (1 -
k / 100));
    },
    fromRgb([r, g, b]) {
        const k = 1 - Math.max(r, g, b) / 255;
        const channel = x => (k === 1 ? 0 : ((1 - x / 255 - k) /
(1 - k)) * 100);
        return [channel(r), channel(g), channel(b), k * 100];
    }
},
HLS: {
    labels: ["H", "L", "S"],
    max: [360, 100, 100],
    toRgb([h, l, s]) {
        l /= 100;
        s /= 100;
        const a = s * Math.min(l, 1 - l);
        const channel = n => {
            const k = (n + h / 30) % 12;
            return 255 * (l - a * Math.max(-1, Math.min(k - 3, 9 -
k, 1)));
        };
        return [channel(0), channel(8), channel(4)];
    },
    fromRgb([r, g, b]) {
        [r, g, b] = [r, g, b].map(x => x / 255);
        const max = Math.max(r, g, b);
        const min = Math.min(r, g, b);
        const d = max - min;
        const l = (max + min) / 2;
        const s = d === 0 ? 0 : d / (1 - Math.abs(2 * l - 1));
        let h = 0;
        if (d !== 0) {
            if (max === r) h = 60 * (((g - b) / d + 6) % 6);
            else if (max === g) h = 60 * ((b - r) / d + 2);
            else h = 60 * ((r - g) / d + 4);
        }
        return [h, l * 100, s * 100];
    }
}
};

const colorBar = document.getElementById("colorBar");
const wheel = document.getElementById("wheel");
const context =
document.getElementById("canvas").getContext("2d");
const hueMarker = document.getElementById("hueMarker");
const colorMarker = document.getElementById("colorMarker");
const container = document.getElementById("models");
const fields = {};
let dragTarget = null;

const SIZE = canvas.clientHeight;
const SCALE = 1;

```

```

const CENTER = SIZE / 2;
const OUTER = 116;
const INNER = 90;
const TRIANGLE = 82;

function toHex(rgb) {
    return "#" + rgb.map(x =>
Math.round(x).toString(16).padStart(2, "0")).join("");
}

function hlsToHsv([h, l, s]) {
    l /= 100;
    s /= 100;
    const v = l + s * Math.min(l, 1 - l);
    return [h, v === 0 ? 0 : 2 * (1 - l / v), v];
}

function hsvToHls([h, s, v]) {
    const l = v * (1 - s / 2);
    const m = Math.min(l, 1 - l);
    return [h, l * 100, m === 0 ? 0 : ((v - l) / m) * 100];
}

function pureColor(h) {
    return MODELS.HLS.toRgb([h, 50, 100]);
}

function angleOfHue(h) {
    return ((h - 180) * Math.PI) / 180;
}

function triangleVertices() {
    return [30, 150, 270].map(degrees => {
        const angle = (degrees * Math.PI) / 180;
        return [CENTER + TRIANGLE * Math.cos(angle), CENTER +
TRIANGLE * Math.sin(angle)];
    });
}

function triangleWeights(x, y, [a, w, k]) {
    // det = (w - k) \dot (a - k)
    // wa = (w - k) \dot ((x, y) - k) / det
    // ww = (a - k) \dot ((x, y) - k) / det

    const det = (w[1] - k[1]) * (a[0] - k[0]) + (k[0] - w[0]) *
(a[1] - k[1]);
    const wa = ((w[1] - k[1]) * (x - k[0]) + (k[0] - w[0]) * (y
- k[1])) / det;
    const ww = ((k[1] - a[1]) * (x - k[0]) + (a[0] - k[0]) * (y
- k[1])) / det;

    return [wa, ww, 1 - wa - ww];
}

```



```

function drawWheel(h) {
  const pixels = SIZE * SCALE;
  const image = context.createImageData(pixels, pixels);
  const vertices = triangleVertices();
  const pure = pureColor(h);
  for (let y = 0; y < pixels; y++) {
    for (let x = 0; x < pixels; x++) {
      const pointX = (x + 0.5) / SCALE;
      const pointY = (y + 0.5) / SCALE;
      const dx = pointX - CENTER;
      const dy = pointY - CENTER;
      const distance = Math.hypot(dx, dy);
      let color = null;
      if (distance >= INNER && distance <= OUTER) {
        const angle = (Math.atan2(dy, dx) * 180) / Math.PI
+ 180;
        color = pureColor((angle + 360) % 360);
      } else {
        const [wa, ww, wk] = triangleWeights(pointX, pointY,
vertices);
        if (wa >= 0 && ww >= 0 && wk >= 0) {
          color = pure.map(channel => wa * channel + ww * 255);
        }
      }
      if (color) {
        image.data.set([...color, 255], (y * pixels + x) * 4);
      }
    }
  }
  context.putImageData(image, 0, 0);
}

function placeMarker(marker, [x, y]) {
  marker.style.left = x + "px";
  marker.style.top = y + "px";
}

function paintWheel() {
  const [h, s, v] = hlsToHsv(readValues("HLS"));
  const [a, w, k] = triangleVertices();
  const weights = [s * v, v * (1 - s), 1 - v];
  const ringRadius = (INNER + OUTER) / 2;
  drawWheel(h);
  placeMarker(hueMarker, [
    CENTER + ringRadius * Math.cos(angleOfHue(h)),
    CENTER + ringRadius * Math.sin(angleOfHue(h))
  ]);
  placeMarker(colorMarker, [
    weights[0] * a[0] + weights[1] * w[0] + weights[2] * k[0],
    weights[0] * a[1] + weights[1] * w[1] + weights[2] * k[1]
  ]);
}

function show(name, values) {

```

```

        fields[name].forEach((field, i) => {
            const value = Math.round(values[i]);
            field.range.value = value;
            field.number.value = value;
        });
    }

    function update(source, values) {
        const rgb = MODELS[source].toRgb(values).map(x => Math.min(255,
Math.max(0, x)));
        for (const name in MODELS) {
            show(name, name === source ? values :
MODELS[name].fromRgb(rgb));
        }
        colorBar.style.background = toHex(rgb);
        paintWheel();
    }

    function readValues(name) {
        return fields[name].map(field => Number(field.number.value));
    }

    function createRow(name, index) {
        const model = MODELS[name];
        const row = document.createElement("div");
        row.className = "row";
        row.innerHTML = `<span>${model.labels[index]}</span>
<input type="range" min="0" max="${model.max[index]}" step="1">
    <input type="number" min="0" max="${model.max[index]}"
step="1">`;
        const range = row.querySelector("[type=range]");
        const number = row.querySelector("[type=number]");

        range.addEventListener("input", () => {
            number.value = range.value;
            update(name, readValues(name));
        });
        number.addEventListener("input", () => {
            if (number.value === "") return;
            number.value = Math.min(model.max[index], Math.max(0,
number.value));
            range.value = number.value;
            update(name, readValues(name));
        });

        fields[name].push({range, number});
        return row;
    }

    for (const name in MODELS) {
        const set = document.createElement("fieldset");
        set.innerHTML = `<legend>${name}</legend>`;
        fields[name] = [];
        MODELS[name].labels.forEach((_, i) =>

```

```

set.append(createRow(name, i));
    container.append(set);
}

function pickHue(x, y) {
    const [, s, v] = hlsToHsv(readValues("HLS"));
    const h = ((Math.atan2(y - CENTER, x - CENTER) * 180) /
Math.PI + 180 + 360) % 360;
    update("HLS", hsvToHls([h, s, v]));
}

function pickColor(x, y) {
    const h = readValues("HLS")[0];
    const weights = triangleWeights(x, y, triangleVertices()).map(w
=> Math.max(0, w));
    const total = weights[0] + weights[1] + weights[2];
    const [wa, ww] = weights.map(w => w / total);
    const v = wa + ww;
    update("HLS", hsvToHls([h, v === 0 ? 0 : wa / v, v]));
}

function pick(event) {
    const box = wheel.getBoundingClientRect();
    const x = event.clientX - box.left;
    const y = event.clientY - box.top;
    if (dragTarget === "hue") pickHue(x, y);
    else pickColor(x, y);
}

wheel.addEventListener("pointerdown", event => {
    const box = wheel.getBoundingClientRect();
    const distance = Math.hypot(event.clientX - box.left - CENTER,
event.clientY - box.top - CENTER);
    dragTarget = distance > INNER ? "hue" : "color";
    wheel.setPointerCapture(event.pointerId);
    pick(event);
});

wheel.addEventListener("pointermove", event => {
    if (dragTarget) pick(event);
});

wheel.addEventListener("pointerup", () => (dragTarget = null));
wheel.addEventListener("pointercancel", () => (dragTarget =
null));

    update("RGB", [58, 123, 213]);
</script>
</body>

</html>

```